



OASIS Security Report

Cross-Site Scripting (XSS) Security Analysis

Date: 2026-06-11 10:57:18 — **Model:** mistral:latest

Vulnerability: Cross-Site Scripting (XSS)

Project: clean_contracts

Analysis root: `../clean`

Table of contents

▢ Summary

▢ Statistics

▢ Files

▢ Detailed analysis

▢ Assistant

△ Errors & notes

Summary

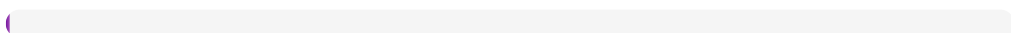
Analyzed **3** file(s) for Cross-Site Scripting (XSS).

[Return to table of contents](#)

Statistics

Files analyzed: 3 **Total findings:** 3

Critical



0

High		0
Medium		3
Low		0

[Return to table of contents](#)

Files

File	Similarity
<code>/work/project/dataset/clean/mycontract_fixed.sol</code>	0.624
<code>/work/project/dataset/clean/modifier_reentrancy_fixed.sol</code>	0.602
<code>/work/project/dataset/clean/simple_dao_fixed.sol</code>	0.588

[Return to table of contents](#)

Detailed analysis

File 1: /work/project/dataset/clean/mycontract_fixed.sol

Similarity score: 0.624

Source lines: 1-31

► Finding 1.1: Vulnerability found (Medium)

Vulnerable code

```
msg.sender == owner
```

Explanation

The 'msg.sender' variable is used to compare with 'owner', which may contain user input. This opens the door for Cross-Site Scripting (XSS) attacks.

Impact

An attacker could potentially steal session cookies, leading to credential theft.

Entry point

sendTo function

Execution path

```
MyContract -> sendTo(...) -> require(msg.sender == owner)
```

Parameters

amount

Exploitation steps

- Send a crafted amount parameter to the 'sendTo' function
- The user input within 'msg.sender' is executed as JavaScript

Example payloads

- `<script>alert('XSS!')</script>`

Remediation

Sanitize all inputs, such as by using context-aware output encoding or Content-Security-Policy.

File 2: /work/project/dataset/clean/modifier_reentrancy_fixed.sol

Similarity score: 0.602

Source lines: 1-42

► Finding 2.1: Vulnerability found (Medium)

Vulnerable code

```
airDrop() supportsToken hasNoBalance public{  
    tokenBalance[msg.sender] += 20;  
}
```

Explanation

The function `airDrop()`, when called, does not sanitize the `msg.sender` input, potentially allowing an attacker to execute XSS attacks.

Impact

Session hijacking or credential theft via XSS payloads

Entry point

`/airDrop`

Execution path

```
***  
user -> airDrop -> tokenBalance[msg.sender]  
***
```

Parameters

`msg.sender`

Exploitation steps

- Call the `airDrop()` function with a crafted XSS payload as the `msg.sender`

Example payloads

- `<script>alert('XSS');</script>`

Remediation

Sanitize the input for `msg.sender` before using it in the `tokenBalance[msg.sender]` assignment.

File 3: /work/project/dataset/clean/simple_dao_fixed.sol

Similarity score: 0.588

Source lines: 1-34

► Finding 3.1: Vulnerability found (Medium)

Vulnerable code

```
'\n * @source: http://blockchain.unica.it/projects/ethereum-survey/
attacks.html#simpledao
 * @author: Atzei N., Bartoletti M., Cimoli T
'
```

Explanation

The code includes user input in the HTML comment section, which could allow an attacker to inject client-side scripts.

Impact

Can lead to session hijacking or delivery of malware to users

Entry point

N/A (comment text)

Execution path

```
user -> comments (injection) -> web page -> victim user
```

Parameters

comments

Exploitation steps

- Attacker injects malicious script in a comment
- Victim visits the webpage with the injected comment
- Script is executed on victim's client

Example payloads

- `<script>alert(document.cookie)</script>`

Conditions

Attacker can control the comments section of the page

Remediation

Sanitize all user-generated HTML content before rendering

Secure example

```
/* comments */
```

[Return to table of contents](#)

Assistant (triage / codebase)

In the OASIS dashboard (JSON report preview), use the chat below to triage findings, exploitation paths, and remediation. Select file, chunk, and finding to narrow context when needed.

[Return to table of contents](#)

Errors & Notes

No file-level errors were recorded.

[Return to table of contents](#)